FACULTAD DE INGENIERÍA Y CIENCIAS

ESCUELA DE INFORMÁTICA Y TELECOMUNICACIONES

Informe de Tarea N.º 02: Torneo de Komikan Concurrente

Sistemas Operativos

Sección: 1

Profesor: Yerko Ortiz

Alumnos: Benjamín Manríquez y Matías Miranda

Índice

1. Diseño del esquema de sincronización **1**
1.1. Arquitectura . . . 1
1.2. Exclusión mutua en el tablero . . . 1
1.3. Juez, mesas y emparejamiento . . . 1
1.4. Fallos y Ctrl+C . . . 1

2. Análisis de riesgos de concurrencia **2**
2.1. Deadlock . . . 2
2.2. Livelock . . . 2
2.3. Starvation . . . 2
2.4. Verificación . . . 2

3. Evaluación **2**

1. Diseño del esquema de sincronización

1.1. Arquitectura

Se usan solo hilos POSIX en un proceso: uno por cada jugador, uno dedicado a `SIGINT` y, por cada mesa activa, un juez y 13 fichas (un puma y 12 perros). La Tabla 1 resume los recursos compartidos.

Recurso compartido	**Primitiva**	**Uso**
Casillas del tablero	`pthread_mutex_t` por casilla	Movimientos
Estado de la partida y log	`mutex_estado`	Fin y contadores
Fin de partida	`cond_fin`	Retardos interrumpibles
Aviso al juez	`cond_juez` + contador	Revisión del tablero
Mesas libres	`sem_t` iniciado en M	Límite y espera pasiva
Sala de espera	mutex y `cond` del servidor	Emparejamiento

Cuadro 1. Recursos compartidos y primitivas de sincronización.

1.2. Exclusión mutua en el tablero

Cada casilla tiene su propio mutex. Antes de moverse, una ficha bloquea su *zona*, las casillas que va a leer o modificar: su casilla y las adyacentes en un perro, y las casillas a distancia 2 en el puma, porque una captura involucra al perro saltado y al destino. Con la zona tomada, la ficha calcula sus movimientos válidos, elige uno y lo aplica en la misma sección crítica, así que nadie puede ocupar el destino entre la decisión y el movimiento. Si dos fichas compiten por una casilla libre, esta pertenece a ambas zonas: la primera que obtiene su mutex se mueve y la segunda la encuentra ocupada. Al capturar, la casilla del perro saltado está en la zona del puma, de modo que el perro no puede moverse durante la captura.

1.3. Juez, mesas y emparejamiento

Tras cada movimiento, la ficha incrementa un contador de eventos y despierta al juez mediante `pthread_cond_signal`. El juez bloquea todas las casillas para obtener una instantánea consistente, registra la matriz en el log y detecta la victoria. Las parejas piden mesa con `sem_wait` sobre un semáforo iniciado en M, que las bloquea sin consumir CPU si no hay mesas libres, y la devuelven con `sem_post`.

1.4. Fallos y Ctrl+C

Un mutex solo puede liberarlo el hilo que lo tomó, por eso cada ficha registra manejadores con `pthread_cleanup_push` que liberan su zona y `mutex_estado` si el hilo termina abruptamente, y avisan al juez. Este declara la victoria del oponente por abandono y verifica con `pthread_mutex_trylock` que ninguna primitiva quedó tomada antes de destruirlas. `SIGINT` se bloquea en todos los hilos y la recibe un hilo dedicado con `sigwait`, que detiene las llegadas e interrumpe las partidas activas.

2. Análisis de riesgos de concurrencia

2.1. Deadlock

Riesgo: dos fichas con zonas traslapadas podrían tomar sus mutex en distinto orden y esperarse mutuamente.

Prevención: existe un orden global: casillas en orden ascendente de índice $(5 \cdot \text{fila} + \text{col})$, luego `mutex_estado`, y el mutex del servidor siempre antes que el de una partida. Todos los hilos lo respetan, incluido el juez al bloquear el tablero completo, por lo que no puede haber espera circular (una de las condiciones de Coffman). Una ficha que falla con mutex tomados también bloquearía a las demás, lo que se evita con los manejadores de limpieza.

2.2. Livelock

Riesgo: fichas que se ceden el paso indefinidamente, por ejemplo con `trylock` y reintentos.

Prevención: no hay reintentos. Una ficha bloquea su zona, decide con información consistente y, si un destino está ocupado, elige otro o espera su siguiente retardo.

2.3. Starvation

Riesgo: los mutex y semáforos POSIX no garantizan orden FIFO. El puma, que bloquea hasta 13 casillas, podría perder repetidamente frente a perros con zonas menores, y una pareja podría esperar mesa más que otras posteriores.

Mitigación: las secciones críticas duran microsegundos y los retardos aleatorios desincronizan a los hilos. Las parejas son finitas y toda partida termina, así que cada una obtiene mesa en algún momento.

2.4. Verificación

Con retardos de 0 a 2 ms (máxima competencia), ThreadSanitizer y `helgrind` no detectaron condiciones de carrera. Un programa independiente reprodujo cada log: todos los movimientos fueron válidos y cada tablero registrado coincidió con el estado real. `valgrind` no reportó fugas ni descriptores abiertos, incluso con Ctrl+C.

3. Evaluación

La Tabla 2 compara la solución con dos alternativas consideradas.

Alternativa	**Complejidad**	**Concurrencia**	**Mov/s***
Un mutex por tablero	Baja	Una ficha a la vez	$\approx 40\,000$
Juez que serializa solicitudes	Media	Una ficha a la vez	No implementada
Mutex por casilla (elegida)	Alta	Zonas disjuntas en paralelo	$\approx 49\,000$

Cuadro 2. Alternativas de diseño. *Movimientos por segundo con una mesa, retardos de 0 ms, promedio de 5 ejecuciones en 4 núcleos.

Con mutex por casilla, fichas en zonas disjuntas se mueven en paralelo, lo que da el mayor nivel de concurrencia y representa mejor la competencia sin turnos. El costo es la complejidad: calcular zonas, respetar el orden global y que el puma tome hasta 13 mutex por movimiento. La ganancia de rendimiento medida es moderada: sin retardos, cerca de un 20 % más de movimientos por segundo que un mutex único, con una y con cuatro mesas ($\approx 92\,000$ frente a $\approx 78\,000$ mov/s). Con los retardos reales de 50 a 300 ms ambas rinden igual, porque la espera domina sobre las secciones críticas. En latencia, lo clave es que los retardos usan `pthread_cond_timedwait`: el fin de la partida o un Ctrl+C despiertan a las fichas de inmediato. Se priorizó así la concurrencia sobre la simplicidad, controlando el riesgo de deadlock con un orden global de bloqueo verificado experimentalmente.